

CI/CD Observability in Containerized Environments

Eduardo Sousa

**A dissertation proposal work plan for the degree of
of Master of Science, Specialisation Area of Cybersecurity And
Systems Administration**

**Advisor: Dr. Alexandre Bragança
Supervisor: Dr. Joao Silva**

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore the work presented in this document is original and authored by me, having not previously been used for any other end.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, Porto, January 6, 2026

Abstract

Internal CI/CD services (GitLab, Jenkins, Nexus, SonarQube, Dependency-Track) often run in Docker on mixed Linux/Windows hosts without administrative access. In these constrained environments, teams struggle to detect faults quickly and to diagnose root causes because telemetry is fragmented and host-level instrumentation is not permitted. This work proposes a container-first observability architecture that collects metrics, logs and traces from non-privileged containers using sidecar exporters, agent collectors and runtime APIs, and correlates signals in a unified metrics pipeline with dashboards and alerts. The approach is validated on a four-server, on-premise setup and evaluated with MTTD, MTTR, pipeline throughput, alert precision and overhead (target below 5%). The expected contribution is a reference architecture, reproducible prototype and decision guidance that improve CI/CD reliability under strict privilege constraints. **Keywords:** CI/CD, Observability, Containers,

Tracing, Logging, Monitoring

Resumo

Trabalhos escritos em língua Inglesa devem incluir um resumo alargado com cerca de 1000 palavras, ou duas páginas.

Se o trabalho estiver escrito em Português, este resumo deveria ser em língua Inglesa, com cerca de 200 palavras, ou uma página.

Para alterar a língua basta ir às configurações do documento no ficheiro `main.tex` e alterar para a língua desejada ('english' ou 'portuguese')¹. Isto fará com que os cabeçalhos incluídos no template sejam traduzidos para a respetiva língua.

¹Alterar a língua requer apagar alguns ficheiros temporários; O target **clean** do **Makefile** incluído pode ser utilizado para este propósito.

Contents

List of Figures	xi
List of Tables	xiii
List of Symbols	xv
1 Introduction	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.2.1 Extended abstract summary	1
1.2.2 Contributions	2
1.3 Research Objectives	2
1.4 Research Questions	2
1.5 Methodology Overview	3
1.6 Ethical Considerations	3
1.7 Structure of the Dissertation	4
2 Literature review	5
2.1 PRISMA Methodology Overview	5
2.1.1 Identification	6
2.1.2 Screening	6
2.1.3 Eligibility	6
2.1.4 Inclusion and Data Extraction	6
2.2 Prisma Flow Diagram	6
2.2.1 Quality assessment criteria	6
2.3 Related Work on Observability in CI/CD Environments	7
2.4 Answering the Research Question	7
2.5 Instrumentation and Deployment Assumptions in the Literature	7
2.6 Performance Evaluation Practices and Reported Overheads	8
2.7 Discrepancies Between Research Environments and Constrained CI/CD Con- texts	9
2.8 Practitioner-Oriented Approaches and Industry Practices	10
2.9 Identified Gaps and Implications for Constrained CI/CD Observability	11
2.10 Summary and Implications for Subsequent Research	12
3 Skill Management and Project Planning	15
3.1 Skill Management	15
3.1.1 Skills Identification	15
3.1.2 Skills Assessment and Development	15
3.1.3 Strategy to Improve Skills	15
3.2 Project Management	16

3.2.1	Phase 1 – PREPD deliverable (November 2025–January 2026)	16
3.2.2	Work Plan (extended abstract)	16
3.2.3	Phase 2 – Thesis development (January–June 2026)	17
3.2.4	Project Charter, Scope and Schedule	17
3.2.5	Monitoring and Controlling Procedures	17
3.2.6	Risk Identification and Management	17
4	Conclusion and Next Steps	19
4.0.1	Validation targets from the extended abstract	20
Appendix A	Appendix Title Here	21

List of Figures

List of Tables

List of Symbols

a	distance	m
P	power	W (Js^{-1})
ω	angular frequency	rad

Chapter 1

Introduction

1.1 Context and Motivation

Internal continuous integration and continuous delivery (Continuous Integration / Continuous Delivery (CI/CD)) services such as GitLab, Jenkins, SonarQube, Nexus and DependencyTrack are central components of many software delivery pipelines. Failures or degraded performance in these services reduce development velocity and can delay releases. This dissertation examines practical observability approaches for containerized CI/CD deployments operating under restricted administrative privileges and in heterogeneous operating environments (mixed Linux and Windows hosts).

The motivation for focusing on constrained, nonprivileged deployments arises from repeated operational observations in industry settings where teams manage their own onpremise infrastructure but lack host administrative access. In such contexts, conventional cloudnative approaches — for example, using privileged host exporters or privileged DaemonSets in Kubernetes — may be unavailable. The research therefore prioritises deployment patterns and tooling that can be applied without modifying service images or requiring host admin rights.

1.2 Problem Statement

The specific research problem addressed in this work is: how to design and implement an integrated observability framework for containerized CI/CD platforms operating under nonprivileged access constraints and heterogeneous infrastructure, such that the resulting artifact supports reproducible evaluation of reliability and incident response improvements.

1.2.1 Extended abstract summary

Internal CI/CD services (GitLab, Jenkins, Nexus, SonarQube, DependencyTrack) often run in Docker containers on mixed Linux/Windows hosts without administrative access. In these constrained environments, teams commonly face difficulty detecting faults promptly and diagnosing root causes because telemetry is fragmented and hostlevel instrumentation is not permitted. This dissertation proposes a containerfirst observability architecture that collects metrics, logs and traces from nonprivileged containers using sidecar exporters, agent collectors and runtime APIs, and correlates signals in a unified telemetry pipeline with dashboards and alerts. The approach will be validated on a fourserver, onpremise deployment (three Linux, one Windows) and evaluated using Mean Time To Detection (MTTD), Mean Time To Recovery (MTTR), pipeline throughput, alert precision and instrumentation overhead

(operational target: below 5%). The expected contribution is a documented reference architecture, a reproducible prototype and decision guidance to improve CI/CD reliability under strict privilege constraints.

1.2.2 Contributions

The primary contributions expected from this research are:

- A containerfirst reference architecture for collecting metrics, logs and traces from nonprivileged containers in mixed OS onpremise environments.
- A reproducible prototype (container images and deployment templates) demonstrating feasible instrumentation patterns (sidecars, collectors, log forwarding) that do not require host administrative privileges.
- An empirical evaluation reporting measured changes in MTTD, MTTR, pipeline throughput and measured instrumentation overhead, including a documented measurement methodology to support reproducibility.
- Practical decision guidance and configuration artefacts to support adoption by operations teams that cannot modify service images or obtain host administrative access.

1.3 Research Objectives

The research objectives are:

- **O1 Reference architecture:** Define a containerfirst reference architecture specifying exporters, collectors, storage backends and visualization components deployable without host administrative privileges.
- **O2 Prototype implementation:** Produce a reproducible prototype (container images and deployment templates) that instruments representative CI/CD services in the target environment.
- **O3 Quantitative evaluation:** Define and execute an empirical evaluation comparing MTTD, MTTR, telemetry pipeline throughput and instrumentation overhead before and after deployment, with documented measurement procedures.
- **O4 Performance constraints:** Characterize resource overhead and document trade-offs; the evaluation includes targets and thresholds (for example, an operational target of under 5% resource overhead) to guide assessment.
- **O5 Reproducibility and guidance:** Provide configuration templates, deployment scripts and adoption guidance for constrained, onpremise environments.

Each objective is scoped to be actionable and measurable. For example, Objective O3 includes explicit measurement procedures (baseline collection, controlled fault injection and paired comparisons) to enable reproducible evaluation rather than solely qualitative assessment.

1.4 Research Questions

The dissertation is guided by the following questions:

1. How can metrics, logs and distributed traces be collected from containerized CI/CD services when only nonprivileged containers are available (no host administrative access)?
2. Which instrumentation and deployment patterns (sidecars, agent collectors, push vs pull pipelines) best support crossserver correlation in mixed OS environments while maintaining acceptable resource overhead?
3. What are the practical constraints and tradeoffs (data fidelity, retention, network and CPU overhead, security) introduced by nonprivileged telemetry collection, and how can they be mitigated in a reproducible reference architecture?

1.5 Methodology Overview

This research follows a design science approach [hevner2004design](#), combining a systematic literature review, comparative technology assessment, artifact design, prototype implementation and empirical evaluation. The work proceeds in five phases:

Phase 1 – Problem Analysis and Literature Review: A PRISMA-guided systematic review of observability frameworks, containerized monitoring and CI/CD reliability to identify approaches and gaps relevant to constrained environments.

Phase 2 – Technology Assessment: Evaluate candidate tools (for example, Prometheus [prometheus2016](#), Loki, Jaeger, OpenTelemetry) against criteria including nonprivileged execution, cross-platform support and resource overhead.

Phase 3 – Architecture Design: Specify component placement (exporters, collectors, storage and visualization), data flows and correlation mechanisms, producing deployment diagrams and configuration templates.

Phase 4 – Prototype Implementation: Implement and document a reproducible prototype using sidecars and collector containers where appropriate; provide deployment automation and configuration artifacts.

Phase 5 – Evaluation and Validation: Quantitatively evaluate MTTD, MTTR and instrumentation overhead; perform controlled fault injection experiments and collect qualitative feedback from practitioners.

The methodological choices reflect the dual goals of creating a practical artefact (the reference architecture and prototype) and generating evaluative evidence about its efficacy. Quantitative evaluation is complemented by qualitative data from practitioner interviews to capture operational acceptability and deployment constraints that are difficult to reproduce in laboratory conditions.

1.6 Ethical Considerations

Human subjects activity (structured interviews) and access to operational telemetry will follow approved protocols. Participants provide informed consent; data are anonymized and handled according to institutional and organizational policies. Production deployments will be preceded by staging tests and changemanagement procedures to avoid operational disruption.

The study will also observe data minimisation principles for telemetry collection and will avoid collecting personal data unless explicitly required and justified; any such collection will follow institutional review procedures and applicable regulations. When presenting empirical results, host identifiers and other sensitive configuration details will be anonymised to protect organisational confidentiality.

1.7 Structure of the Dissertation

The remainder of the dissertation is organised as follows: Chapter 2 presents project planning and skill management; Chapter 3 describes the systematic literature review and related work; Chapter 4 summarises conclusions and outlines next steps for prototype implementation and evaluation. Appendices provide supplementary materials and configuration artifacts.

Chapter 2

Literature review

2.1 PRISMA Methodology Overview

A systematic literature review will be conducted following Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) guidelines to ensure comprehensive and reproducible coverage of the research domain. The search strategy encompasses the following elements:

Information sources:

- **Academic databases:** IEEE Xplore, ACM Digital Library, Springer, ScienceDirect, and Google Scholar for peer-reviewed publications
- **Technical repositories:** arXiv for preprints and working papers in systems and software engineering
- **Gray literature:** Technical documentation from established open-source observability projects, industry whitepapers, and practitioner reports from reputable sources (given the rapid evolution of container observability practices)

Search terms and queries: Core search queries will combine terms from three concept groups using Boolean operators:

1. *Observability concepts:* “observability”, “monitoring”, “telemetry”, “instrumentation”, “metrics”, “logging”, “tracing”, “distributed tracing”
2. *Container and deployment context:* “container”, “containerized”, “Docker”, “non-privileged”, “unprivileged”, “sidecar”, “agent”
3. *Application domain:* “CI/CD”, “continuous integration”, “continuous deployment”, “DevOps”, “reliability”, “incident response”, “MTTD”, “MTTR”

Example query structure: (*observability* OR *monitoring* OR *telemetry*) AND (*container* OR *containerized* OR *non-privileged*) AND (*CI/CD* OR *reliability* OR *incident response*)

Inclusion and exclusion criteria:

- **Inclusion:** Peer-reviewed publications from 2018–2025, technical reports and documentation addressing container telemetry collection, instrumentation patterns, performance evaluation, or reliability improvement in constrained or heterogeneous environments, comparative studies of observability approaches, and empirical evaluations of monitoring overhead

- **Exclusion:** Publications exclusively focused on privileged host access patterns without discussion of constraints, cloud-provider-specific solutions that cannot generalize to on-premise infrastructure, purely theoretical work without practical implementation guidance, and papers predating 2018 (unless seminal works establishing foundational concepts)

Selection process: Literature selection will follow a three-stage process: (1) title and abstract screening to identify potentially relevant sources, (2) full-text review to assess relevance against inclusion/exclusion criteria, and (3) quality assessment focusing on methodological rigor, reproducibility, and applicability to constrained environments. Reference chaining (backward and forward citation tracking) will supplement database searches to identify additional relevant work.

2.1.1 Identification

The identification stage will record search queries, databases queried, and the number of retrieved records. Queries will be versioned and documented to enable replication of the search process.

2.1.2 Screening

Title and abstract screening will be performed using documented inclusion/exclusion criteria; screening decisions and reasons for exclusion at this stage will be recorded. When uncertain, items will be advanced to fulltext review to avoid premature exclusion.

2.1.3 Eligibility

Fulltext review assesses whether the study meets the inclusion criteria and whether it reports sufficient methodological detail for quality assessment. Studies that lack sufficient detail will be noted but may be excluded from quantitative synthesis.

2.1.4 Inclusion and Data Extraction

Included studies will have data extracted into structured evidence tables. Extraction fields include: study context, deployment environment (cloud/onpremise), privilege assumptions, telemetry modalities used, evaluation metrics and experimental design. Extraction will be doublechecked for accuracy.

2.2 Prisma Flow Diagram

The PRISMA flow diagram will be generated from the recorded counts at each stage (identification, screening, eligibility, inclusion) and included in the thesis as a visual summary of the search process.

2.2.1 Quality assessment criteria

Quality assessment will evaluate internal validity (study design, controls), external validity (generalizability to constrained CI/CD environments), and reporting transparency (availability of artifacts, measurement procedures). Assessment outcomes will guide whether a study contributes to quantitative synthesis or only qualitative insight.

2.3 Related Work on Observability in CI/CD Environments

The literature review will examine existing approaches to container observability, deployment patterns for telemetry collection (agent-based, sidecar, host-level), instrumentation frameworks and their privilege requirements, performance and overhead studies, and empirical evaluations of monitoring interventions.

Preliminary reading indicates that much of the documented practice and empirical evaluation in the literature assumes elevated privileges, homogeneous cloud environments and a high degree of control over service code and deployment (for example, the use of sidecar patterns and Kubernetes operators). There is comparatively less empirical guidance for internal, on-premise CI/CD deployments where host administration is restricted and thirdparty packaged services cannot be modified.

Below we summarise representative studies and identify areas where evidence is limited or absent. These summaries are intended to guide the systematic review rather than to be exhaustive.

2.4 Answering the Research Question

The systematic review will produce an evidence base to answer the research questions by: (a) identifying feasible telemetry collection patterns under privilege constraints; (b) documenting evaluation methods and KPIs suitable for constrained environments; and (c) providing references and examples to inform the prototype implementation and measurement strategy.

2.5 Instrumentation and Deployment Assumptions in the Literature

The comparative analysis of published observability systems reveals a consistent pattern of deployment and privilege assumptions that influence the applicability of proposed solutions to constrained environments. A substantial proportion of evaluated architectures presume host-level administrative access, either through privileged containers with direct socket access, kernel-level instrumentation frameworks, or cluster-wide operator deployments. Correia et al. **correiaMonintainerOrchestrationindependentExtensible2023** exemplify this pattern with a hybrid push-pull architecture requiring node-level agents with `cgroup` access for container metrics and RAPL interfaces for energy profiling, capabilities unavailable without elevated privileges. Similarly, Dinga et al. **dingaEmpiricalEvaluationEnergy2023** deploy host-based collectors such as Metricbeat and cAdvisor with direct Docker API access, supplemented by application-level Zipkin tracing that necessitates explicit integration with containerized services. These deployment patterns, while effective in homogeneous cloud platforms where orchestrators manage infrastructure, present significant barriers in heterogeneous or restricted environments where node access is administratively controlled.

A second class of instrumentation approaches leverages kernel observability mechanisms, particularly extended Berkeley Packet Filter (eBPF), to achieve non-intrusive telemetry collection. Works such as Sharma et al. **sharmaEBPFEnhancedCompleteObservability2024** and Kannan et al. **kannanDesigningLightweightNetwork2024** demonstrate that kernel-level probes can extract rich observability signals—including network flow metrics, system

call traces, and container-level resource utilization—without requiring application code modifications. However, these mechanisms impose their own set of deployment prerequisites: Linux kernel versions that support stable eBPF interfaces (typically v5.15 or newer), capabilities such as `CAP_BPF` and `CAP_NET_ADMIN`, and deployment as privileged DaemonSets with node-wide scope. While such techniques offer substantial reductions in instrumentation overhead compared to sidecar proxies or per-service agents, they remain inapplicable in scenarios where host kernel access is restricted or where execution occurs on heterogeneous platforms without uniform kernel feature availability. The distinction between operational feasibility and privilege requirements emerges as a critical dimension in assessing the transferability of observability architectures to environments lacking administrative control over underlying infrastructure.

Sidecar-based collection represents a third architectural family, evaluated extensively in service mesh contexts by Hasanath et al. **hasanthEvaluatingPerformanceSidecarbased2024** and Salcedo-Navarro et al. **salcedo-navarroK8sidecarModularKubernetes2025**. Sidecars intercept network traffic at the pod boundary, enabling transparent telemetry collection without modifying application logic. Yet sidecar deployments introduce resource multipliers proportional to service count—each sidecar consumes CPU, memory, and network bandwidth—and rely on orchestrator-managed injection mechanisms that may not be available or permissible in restrictive CI/CD environments. Furthermore, latency analysis by Sahu et al. **sahuSidecarsCentralLane2023** indicates measurable microarchitectural contention between application and sidecar containers sharing vCPU resources, underscoring that deployment simplicity does not eliminate performance implications.

2.6 Performance Evaluation Practices and Reported Overheads

Empirical evaluations in the reviewed literature exhibit considerable heterogeneity in experimental design, workload selection, and performance metrics, complicating direct comparisons across studies. Benchmark applications vary from synthetic microservice suites such as Online Boutique **hasanthEvaluatingPerformanceSidecarbased2024** and Death-StarBench **TraceWeaverDistributedRequest** to production-scale workloads deployed in commercial platforms **liuJCallGraphTracingMicroservices2019**. Load generation techniques range from controlled concurrency ramps using tools such as Locust to stochastic arrival models with diverse traffic patterns. Overhead is quantified through disparate metrics—CPU utilization percentages, absolute memory footprints, throughput degradation relative to baseline, and latency percentiles—reported at different granularities (per-node, per-container, system-wide aggregates). This methodological diversity reflects the absence of standardized observability benchmarking protocols for containerized systems, a gap that hinders reproducibility and cross-study synthesis.

Reported overhead magnitudes span a wide range depending on instrumentation technique and signal fidelity. Host-based eBPF solutions demonstrate notably lower resource consumption than alternative approaches: Sharma et al. **sharmaEBPFEnhancedCompleteObservability2024** report CPU usage reductions of twenty-one to two-hundred-fourteen times compared to OpenTelemetry agents, sidecar proxies, and traditional host exporters. Kannan et al. **kannanDesigningLightweigh** document approximately ten percent throughput degradation and point-five percent per-node CPU overhead for network flow monitoring via Traffic Control hooks, contrasting favorably with legacy packet capture tools that impose fifty to seventy percent throughput penalties.

Conversely, full-stack distributed tracing through export-based OpenTelemetry collectors incurs substantial costs: performance evaluations on serverless and microservice workloads reveal throughput reductions ranging from nineteen to eighty percent and latency increases between seven and one-hundred-seventy-five percent **InvestigatingPerformanceOverhead**. Tail-based sampling strategies, while improving trace completeness, amplify collector CPU consumption by seventy-one percent relative to head-based sampling **karkanPerformanceOverheadOpenTel**. These figures underscore the existence of fundamental tradeoffs between signal completeness, collection latency, and resource overhead, tradeoffs that must be navigated in resource-constrained deployment contexts.

Several studies acknowledge methodological limitations that restrict the generalizability of overhead assessments. Dinga et al. **dingaEmpiricalEvaluationEnergy2023** note the inability to isolate container-level energy consumption using external power measurement, attributing observed increases to host system-wide effects rather than application-specific telemetry costs. Norgren **norgrenOPTIMIZINGDISTRIBUTEDTRACING** highlights that collector placement decisions—DaemonSet versus sidecar—interact with orchestrator resource limits and TraceContext propagation overhead in ways that are not easily predicted from component-level microbenchmarks. Ramachandran et al. **ramachandranFUSEFaultDiagnosis2023** document latency amplifications between thirteen and forty-five times baseline when employing comprehensive eBPF syscall tracing for fault diagnosis, demonstrating that even low-overhead kernel mechanisms impose non-negligible penalties when probe invocation frequency is high. The variability of reported overheads reflects not only differences in experimental setup but also the sensitivity of performance outcomes to workload characteristics, instrumentation scope, and backend processing efficiency.

2.7 Discrepancies Between Research Environments and Constrained CI/CD Contexts

The operational assumptions underlying much of the published research diverge substantially from the constraints characteristic of on-premise CI/CD deployments in heterogeneous or administratively restricted environments. The majority of evaluated systems presume orchestrator-mediated infrastructure management, often Kubernetes, with the authority to deploy cluster-wide resources such as DaemonSets, CustomResourceDefinitions, and admission controllers. This model affords centralized configuration, uniform telemetry pipelines, and the ability to inject sidecars or agents into workloads declaratively. By contrast, CI/CD systems operating on shared infrastructure with limited user privileges cannot rely on such orchestrator capabilities. Executors may run as unprivileged containers without access to host namespaces, kernel interfaces, or Docker sockets, precluding the direct application of host-based or kernel-level instrumentation techniques documented in the literature.

Furthermore, research deployments typically assume homogeneous execution environments—identical kernel versions, uniform container runtimes, consistent network plugins—facilitating predictable instrumentation behavior. Real-world CI/CD infrastructure, particularly in organizations with diverse legacy systems or multi-cloud deployments, exhibits platform heterogeneity: varying operating system distributions, mixed kernel versions, and differing container runtime policies. Observability mechanisms dependent on specific kernel features, such as eBPF tracepoints introduced in recent Linux releases, fail silently or degrade gracefully depending on fallback support, introducing operational fragility. The assumption of code modifiability presents another divergence: academic prototypes often demonstrate instrumentation

by integrating tracing libraries into purpose-built microservice applications, whereas CI/CD pipelines frequently execute third-party packaged software—build tools, testing frameworks, dependency resolvers—that cannot be recompiled or instrumented without violating support contracts or introducing version drift.

A third area of misalignment concerns the evaluation of correlation mechanisms across observability signals. Research systems implementing distributed tracing commonly propagate trace context through HTTP headers or gRPC metadata, mechanisms that require either application-level instrumentation or transparent proxy interception at service boundaries. In CI/CD contexts, correlation must often be reconstructed post hoc from logs, metrics, and execution metadata available through pipeline orchestration APIs, without the benefit of in-band trace propagation. This retrospective correlation problem—linking container resource spikes to specific job identifiers, associating log error patterns with pipeline failure events—remains underexplored in the literature, which tends to assume instrumentation frameworks designed for long-lived service deployments rather than ephemeral task-based workloads. The temporal and architectural characteristics of CI/CD executions—short-lived containers, stateless task execution, batch job semantics—differ meaningfully from the microservice interaction patterns that dominate the observability research corpus.

2.8 Practitioner-Oriented Approaches and Industry Practices

Beyond the academic literature, industry documentation and technical reports from observability platform vendors provide complementary perspectives on telemetry collection in production environments. These sources emphasize operational pragmatism, configuration flexibility, and the incremental adoption of observability capabilities within existing infrastructure constraints. For instance, OpenTelemetry project documentation outlines collector deployment patterns ranging from agent-per-host configurations to centralized gateway architectures, explicitly addressing tradeoffs between local buffering, network bandwidth consumption, and backend ingestion capacity. Prometheus operational guidelines recommend pull-based metric collection with service discovery mechanisms that decouple telemetry exporters from backend topology, enabling declarative configuration suitable for dynamic containerized environments. Grafana Loki documentation describes log aggregation without indexing full message content, reducing storage overhead at the cost of query expressiveness, a tradeoff relevant to resource-constrained deployments.

Vendor technical whitepapers and case studies, while less methodologically rigorous than peer-reviewed research, document deployment experiences across diverse organizational contexts. These reports frequently highlight challenges not emphasized in academic work: authentication and authorization models for multi-tenant telemetry pipelines, data retention policies balancing compliance requirements against storage costs, and observability tool sprawl arising from heterogeneous technology stacks. Datadog and Dynatrace technical blogs discuss the operational complexity of maintaining unified observability across hybrid cloud and on-premise infrastructure, noting that privilege limitations in customer-managed environments necessitate agent deployment models that sacrifice telemetry completeness for installation simplicity. Elastic observability architecture documentation describes hybrid push-pull patterns where agents push metrics and logs to centralized endpoints, accommodating firewall traversal and egress-only network policies common in enterprise settings.

Industry practices also reveal the prevalence of logging-centric observability as a pragmatic compromise when comprehensive instrumentation is infeasible. Structured logging with correlation identifiers, combined with centralized log aggregation and pattern matching, provides limited but actionable visibility without requiring application code changes or privileged host access. Jenkins and GitLab CI/CD platform documentation emphasizes log-based troubleshooting workflows, reflecting the operational reality that pipeline executions are often diagnosed through console output rather than through distributed traces or granular metrics. This logging-first approach aligns with the ephemeral nature of CI/CD workloads and the difficulty of instrumenting arbitrary third-party build tools. However, the literature provides minimal empirical guidance on the diagnostic effectiveness of log-only observability compared to multi-signal approaches, leaving a gap between documented best practices and evidence-based performance validation.

2.9 Identified Gaps and Implications for Constrained CI/CD Observability

The synthesis of reviewed literature reveals several areas where existing research provides limited guidance for observability in privilege-constrained, heterogeneous CI/CD environments. First, empirical studies evaluating non-privileged telemetry collection patterns remain scarce. While sidecar-based approaches nominally operate without host-level privileges, performance evaluations typically occur in Kubernetes clusters where orchestrator-managed injection and resource allocation mitigate operational complexity. The feasibility and overhead characteristics of sidecar telemetry in environments where container orchestration is absent or restricted—such as standalone Docker executor environments or VM-based runners—have not been systematically characterized. Similarly, the applicability of application-level instrumentation libraries to third-party packaged software, a common scenario in CI/CD pipelines, receives minimal attention in the literature, which predominantly evaluates custom-built microservice applications amenable to source-level modification.

Second, the literature offers limited insight into post hoc correlation techniques suitable for ephemeral task-based workloads. Research on distributed tracing emphasizes in-band context propagation through service invocation chains, a model poorly suited to CI/CD executions where observability signals must be correlated retrospectively using pipeline metadata, container identifiers, and temporal alignment of logs and metrics. Techniques for reconstructing causal relationships between resource contention events and pipeline failures, or for attributing aggregate host metrics to individual containerized build tasks, are underexplored. This gap is significant because CI/CD observability fundamentally addresses a diagnostic problem—identifying which pipeline stage failed and why—rather than the performance optimization focus that dominates microservice observability research.

Third, the methodological practices documented in the literature provide insufficient guidance for evaluating observability interventions in controlled CI/CD settings. Standard microservice benchmarks such as Online Boutique or DeathStarBench model synchronous request-response patterns with stateless service replicas, characteristics that diverge from CI/CD workload semantics: batch job execution, stateful build caches, sequential pipeline stages with inter-stage dependencies. Metrics such as MTTD and MTTR, frequently cited as primary outcome measures for observability effectiveness, lack standardized operational definitions in the CI/CD context. Variability in how detection events are defined (alert trigger, operator awareness, automated remediation) and how recovery is measured (pipeline

restart, job completion, service restoration) complicates the interpretation and comparison of reported improvements. The absence of reproducible benchmark artifacts—representative CI/CD workloads, fault injection scenarios, instrumentation templates—further hinders empirical validation of observability claims.

Finally, the literature exhibits limited engagement with operational constraints imposed by multi-tenant shared infrastructure, data residency requirements, and restricted network egress policies. Research prototypes typically assume unimpeded network connectivity to backend observability platforms, whereas enterprise CI/CD environments may mandate on-premise data retention, prohibit external telemetry export, or enforce egress filtering that complicates agent-to-collector communication. The implications of such constraints for collector topology, buffering strategies, and telemetry completeness remain largely unaddressed. These gaps collectively indicate that while the observability research community has produced substantial evidence on instrumentation techniques, performance tradeoffs, and architectural patterns, the direct applicability of these findings to constrained CI/CD contexts requires further investigation and adaptation.

2.10 Summary and Implications for Subsequent Research

This chapter has established the methodological foundation for systematic literature review and synthesized existing research on container observability, with particular attention to deployment assumptions, performance characteristics, and the alignment between documented practices and the operational constraints of privilege-restricted CI/CD environments. The reviewed literature demonstrates that contemporary observability systems achieve rich telemetry collection through diverse instrumentation techniques—host-based kernel probes, sidecar proxies, application-level tracing libraries—each exhibiting distinct privilege requirements, performance overheads, and applicability boundaries. While kernel-level eBPF mechanisms offer substantial efficiency advantages, their dependence on host access and recent kernel features limits transferability to heterogeneous or restricted platforms. Sidecar patterns provide deployment flexibility but introduce resource multipliers and microarchitectural contention. Application-level instrumentation enables fine-grained observability at the cost of code modification and cross-platform compatibility challenges.

The comparative analysis reveals that much of the published research presumes operational capabilities unavailable in constrained CI/CD deployments: administrative host access, orchestrator-mediated resource management, and the ability to modify or instrument application code. Evaluation practices, while methodologically diverse, predominantly target long-lived microservice workloads rather than ephemeral task-based executions characteristic of CI/CD pipelines. Metrics such as MTTD and MTTR, though frequently invoked, lack standardized definitions suitable for comparing observability interventions across heterogeneous systems. These findings underscore the necessity of adapting existing instrumentation techniques and evaluation methodologies to the specific constraints and workload characteristics of on-premise, multi-tenant CI/CD infrastructure.

The identified gaps in existing research inform the design and evaluation strategy for subsequent phases of this work. The absence of empirical studies characterizing non-privileged telemetry collection in CI/CD contexts motivates the development of a prototype observability system that operates within documented privilege boundaries while providing actionable diagnostic signals. The limited guidance on post hoc correlation for ephemeral workloads necessitates the investigation of metadata-based correlation techniques leveraging pipeline

orchestration APIs and temporal alignment heuristics. The methodological gap in controlled CI/CD evaluation frameworks justifies the establishment of reproducible benchmark scenarios with standardized fault injection and measurement procedures. Collectively, the evidence synthesized in this chapter establishes the research context and motivates the technical contributions described in subsequent chapters, bridging the gap between general-purpose observability research and the specific requirements of constrained CI/CD observability.

Chapter 3

Skill Management and Project Planning

3.1 Skill Management

This section summarises the skills required for the research activities and the planned approach for acquiring or consolidating them.

3.1.1 Skills Identification

Key technical skills include container orchestration and debugging, telemetry and observability tooling (OpenTelemetry, Prometheus, Loki, Jaeger), scripting for automation, and experimental design for empirical evaluation. Soft skills include project management and structured interviewing for qualitative validation.

The selection emphasises skills that are directly relevant to the prototype implementation and evaluation: reliable collection of telemetry from containers without elevated privileges, crossplatform troubleshooting (Linux and Windows containers), and reproducible deployment using containers and scripts. Competence in interpreting timeseries data and traces for incident diagnosis is also required to validate the proposed evaluation metrics.

3.1.2 Skills Assessment and Development

An initial self-assessment identifies gaps in advanced tracing and Windows container telemetry; these will be addressed through focused practical exercises, small proof-of-concepts, and targeted readings during Phase 1.

3.1.3 Strategy to Improve Skills

The strategy combines short, time-boxed experiments, community documentation and tutorials, and incremental integration into the prototype. All skill development tasks are recorded and linked to project milestones to ensure traceability.

Planned activities include the following practical exercises:

- Implement a minimal sidecar exporter for a simple containerised service to validate nonprivileged metrics collection.
- Configure Prometheus scraping against nonstandard endpoints and verify collection reliability on both Linux and Windows hosts.

- Perform lightweight tracing with OpenTelemetry to confirm context propagation using proxy or sidecar approaches.

Each exercise has acceptance criteria (e.g. successful metric collection for 24 hours under synthetic load) to provide objective evidence of skill acquisition.

3.2 Project Management

The research timeline comprises two main phases: the PREPD deliverable (November 2025–January 2026) and thesis development (January–June 2026). Each phase includes concrete deliverables and acceptance criteria.

3.2.1 Phase 1 – PREPD deliverable (November 2025–January 2026)

- **Literature review:** Conduct a systematic review using PRISMA to identify observability frameworks and deployment patterns relevant to nonprivileged containers.
- **Technology assessment:** Evaluate candidate technologies and produce a comparative matrix aligned with constraints.
- **Initial architecture design:** Produce a highlevel reference architecture and validate it against constraints.
- **Baseline data collection:** Establish baseline telemetry measurements and measurement procedures.
- **PREPD delivery:** Submit the PREPD report and presentation materials.

3.2.2 Work Plan (extended abstract)

The extended abstract summarises the planned activities and deliverables for the PREPD and thesis phases. It frames Phase 1 as focusing on literature review, technology assessment and baseline data collection, and Phase 2 as addressing prototype implementation, production deployment and evaluation. The present document adopts that plan and expands it with acceptance criteria and measurable outputs.

The main tasks from the extended abstract are reiterated here for traceability:

- **Phase 1 (Nov–Jan):** Systematic literature review using PRISMA; comparative technology assessment; initial architecture design; baseline telemetry collection; PREPD report submission.
- **Phase 2 (Jan–Jun):** Finalise state of the art and select technologies (Jan–Feb); implement prototype and deployment automation (Feb–Mar); coordinate production deployment subject to approvals (Apr); collect observability data and run measurement campaigns (May); validate, analyse and write up results (Jun); final submission (mid-Jun).

Each task includes acceptance criteria such as documented evidence tables for the review, a completed technology assessment matrix, reproducible deployment scripts for the prototype, and an archived dataset with analysis scripts for the evaluation phase.

3.2.3 Phase 2 – Thesis development (January–June 2026)

- **Detailed architecture and prototype implementation (Feb–Mar):** Implement the prototype in staging with reproducible deployment artifacts.
- **Production deployment (Apr):** Coordinate deployment to target servers, following changemanagement procedures.
- **Observability data collection (May):** Execute the measurement campaign and collect quantitative metrics.
- **Validation, analysis and writing (Jun):** Analyze results, collect practitioner feedback and complete thesis chapters.
- **Final submission:** Incorporate feedback and submit the dissertation.

3.2.4 Project Charter, Scope and Schedule

Detailed work breakdowns (WBS), acceptance criteria and the integrated schedule are maintained in project artefacts and updated at milestone reviews.

Key deliverables and acceptance criteria (examples):

- **Deliverable: Reference architecture diagram.** Acceptance: diagram includes component placement, data flows and privilege assumptions, and is reviewed by advisor.
- **Deliverable: Prototype artifacts.** Acceptance: reproducible deployment scripts and container images that can be deployed in a staging environment following the provided README.
- **Deliverable: Evaluation dataset and analysis.** Acceptance: raw telemetry and analysis scripts are archived and sufficient to reproduce primary metrics reported in the thesis.

3.2.5 Monitoring and Controlling Procedures

Progress monitoring: Regular meetings with the advisor and company supervisor and periodic written reports will track milestones and risks.

Risk management: Key risks include infrastructure provisioning delays, limited production access and tooling incompatibilities. Mitigations include extended staging tests, alternative evaluation scenarios using staging data, and early proof-of-concept experiments.

3.2.6 Risk Identification and Management

Risks will be logged in the project risk register, prioritised using an impact/probability matrix, and assigned mitigations and owners. The register will be reviewed at milestone checkpoints and adapted as new information arises.

An illustrative risk table will be maintained in project artefacts; examples of mitigations include the use of staging testbeds to emulate production behaviour and fallback measurement strategies if production deployment is delayed.

Chapter 4

Conclusion and Next Steps

This chapter summarises the dissertation scope and outlines the immediate next steps for prototype implementation and empirical evaluation. The work presented so far defines the problem context, research objectives, methodology and the protocol for a PRISMA-guided literature review. The subsequent phases will apply the evidence collected in Chapter 2 to select technologies, design a reproducible reference architecture and implement a prototype suitable for constrained, nonprivileged CI/CD environments.

Next steps:

- Finalise the systematic literature review and produce the Prisma flow diagram and evidence tables.
- Complete the technology assessment matrix and select the candidate stack for prototyping, documenting selection rationale and compatibility with privilege constraints.
- Implement the prototype in a staging environment, produce deployment templates and instrument the selected CI/CD services following the reference architecture.
- Execute the measurement campaign (baseline and postdeployment), including controlled fault injection experiments, and collect quantitative and qualitative data according to the evaluation plan.
- Analyse results, generalise findings where appropriate, and prepare dissertation chapters describing implementation, evaluation and implications for practice.

The project will avoid making definitive claims about generalisability until the prototype evaluation and analysis are complete; all conclusions will be grounded in the empirical evidence and contextualised relative to the reviewed literature. The following deliverables are expected at the end of the evaluation phase:

- A documented reference architecture, including deployment templates and configuration artifacts.
- A reproducible prototype that can be deployed in a staging environment using provided scripts and container images.
- An evaluation report containing raw measurement data, analysis scripts, and a reproducible pipeline to regenerate key figures and tables.
- A practitioner guidance note summarising deployment constraints, tradeoffs and recommended practices for nonprivileged CI/CD observability.

Timelines and responsibilities for these deliverables are recorded in project artefacts and will be reviewed during milestone meetings. After the evaluation, results will be compared with the literature synthesis to position the contributions and limitations of the work.

Potential dissemination channels include the thesis, technical reports, and possible conference or workshop submissions focusing on observability in constrained environments; decisions on dissemination will observe any organisational publication policies regarding operational data.

4.0.1 Validation targets from the extended abstract

The extended abstract proposes a focused set of evaluation targets that will be used to validate the prototype. These include:

- Primary metrics: reductions in MTDD and MTTR compared to baseline measurements.
- Pipeline throughput: verification that the telemetry pipeline can sustain the required ingestion rate without data loss under expected load profiles.
- Alert precision and detection coverage: measured via controlled fault injection and posthoc analysis of alerting outcomes.
- Instrumentation overhead: quantification of CPU, memory and network overhead introduced by the observability stack, with an operational target of keeping overhead below 5% where feasible.

These targets will be operationalised with specific measurement procedures and acceptance criteria documented in the evaluation plan; conclusions will be restricted to the contexts and environments tested and will not be overgeneralised without supporting evidence.

Appendix A

Appendix Title Here

Write your Appendix content here.